

Bounded Strong-Ring Classification Specification

Stage 7R Revision: Persistent Result, Transient Workspace, and Verifiable Certificates

mdstats

2026-07-18

Contents

Purpose and boundary	1
Mathematical definition	1
Domain and resources	1
Transient search workspace	2
Persistent result	2
Candidate enumeration	3
Exact cancellation and memory preflight	3
Independent verification	3
Public builders	3
Completeness and source constraints	4
Storage scaling	4
Validation requirements	4
References	4

Purpose and boundary

ring_strength.py classifies one primitive-ring placement as weak or boundedly strong inside an explicit finite placement domain.

Runtime/API target:

mdstats 0.19.19a0

Stage 7R separates four different objects:

scientific result $\neq$ derived candidate workspace $\neq$ positive witness $\neq$ execution resources .

The persistent result no longer stores every enumerated candidate placement.

Mathematical definition

For target support $[C]$ and smaller primitive-ring placements R_j , weakness in a finite domain is

$$[C] \in \text{span}_{\text{GF}(2)} \{[R_1], \dots, [R_N]\}.$$

A positive witness satisfies

$$[C] \oplus [R_{j_1}] \oplus \dots \oplus [R_{j_m}] = 0$$

as exact physical lifted-edge incidence.

It suffices to use strictly smaller primitive rings: any nonprimitive smaller cycle decomposes recursively into still smaller cycles, and strict length descent terminates.

Domain and resources

```
@dataclass(frozen=True)
class EdgeIncidencePlacementDomain:
    max_incidence_depth: int
```

```
@dataclass(frozen=True)
class RingStrengthDomain:
    target_ring_key: PrimitiveRingKey
    max_component_size: int
    placement_domain: EdgeIncidencePlacementDomain
```

No `max_component_count` belongs to the mathematical domain. Finite $\text{GF}(2)$ span membership already assigns each candidate coefficient zero or one.

```
@dataclass(frozen=True)
class RingStrengthResources:
    max_candidate_placements: int = 50_000
    max_search_nodes: int = ...
    max_support_terms: int = ...
    max_matrix_bits: int = ...
    max_provenance_bits: int = ...
```

The matrix and provenance-bit limits guard the actual Python-integer elimination representation before allocation.

Transient search workspace

```
@dataclass(frozen=True)
class RingStrengthSearchWorkspace:
    topology_graph_digest: str
    primitive_ring_catalog_digest: str
    target_placement: RingPlacement
    domain: RingStrengthDomain
    resources: RingStrengthResources
    diagnostics: RingStrengthDiagnostics
    candidate_placements: tuple[RingPlacement, ...]
    candidate_set_digest: str
```

The workspace is deterministic but intentionally has no persistent scientific schema. It may be retained temporarily for diagnostics or debugging.

```
build_ring_strength_workspace(
    index,
    target_placement,
    domain,
    *,
    resources=None,
) -> RingStrengthSearchWorkspace
```

Persistent result

```
class RingStrengthStatus(str, Enum):
    WEAK_CERTIFIED = ...
    STRONG_IN_DOMAIN = ...
    UNRESOLVED_TRUNCATED = ...
    UNRESOLVED_SOURCE_INCOMPLETE = ...

@dataclass(frozen=True)
class RingStrengthResult:
    topology_graph_digest: str
    primitive_ring_catalog_digest: str
    target_placement: RingPlacement
    domain: RingStrengthDomain
    resources: RingStrengthResources
    status: RingStrengthStatus
    diagnostics: RingStrengthDiagnostics
    candidate_set_digest: str
    witness: RingStrengthWitness | None
    canonical_schema_version: str
```

```
digest_algorithm: str
digest: str
```

The result records the digest of the exhaustive finite candidate set, not the candidate tuple itself.

For WEAK_CERTIFIED, witness contains the exact component placements. For STRONG_IN_DOMAIN, the negative theorem is scoped only to the declared finite domain. Resource and source failures remain unresolved.

Schema:

```
mdstats.ring_strength.v2
mdstats.ring_strength_catalog.v2
```

Candidate enumeration

Starting from the target's physical lifted edges, the backend explores the bipartite incidence relation

lifted edge instance <-> admissible translated smaller-ring placement

breadth first to the declared incidence depth. Candidate ordering, support ordering, and the candidate_set_digest are deterministic.

Exact cancellation and memory preflight

The workspace is passed to solve_finite_ring_cancellation with FiniteRingCancellationResources. Before elimination, the backend estimates:

$$B_{\text{matrix}} = NE,$$
$$B_{\text{provenance}} = N \min(N, E),$$

where N is the candidate count and E the number of represented physical edge instances. If either declared bit limit is exceeded, classification returns UNRESOLVED_TRUNCATED rather than risking uncontrolled memory growth.

Independent verification

```
result.verify(index)
```

performs source-bound verification. It:

1. checks topology and primitive-ring catalog digests;
2. verifies a weak witness by exact edge parity;
3. deterministically re-enumerates the declared finite domain;
4. confirms the candidate-set digest; and
5. replays the exact finite classification and compares the canonical result.

RingStrengthResult.from_dict(..., index=index, verify=True) and catalog loading perform this verification by default. A recomputed JSON digest alone is not accepted as a scientific certificate.

Public builders

```
classify_ring_strength(
    index,
    target_placement,
    domain,
    *,
    resources=None,
) -> RingStrengthResult

build_ring_strength_catalog(
    index,
    domains,
    *,
    resources=None,
) -> RingStrengthCatalog
```

Completeness and source constraints

- The primitive-ring catalog must be lower closed through `max_component_size`.
- The source search must have completed without resource truncation.
- The target and all candidates must share the exact topology graph digest.
- Every finite-domain negative result is bounded; no result is silently promoted to global strength.
- Unresolved status must propagate to face and tiling certification.

Storage scaling

Persistent storage is now proportional to the target/domain/diagnostics and an optional compact witness. It is no longer proportional to the complete candidate workspace. In the LTA depth-eight fixture, a 3,240-candidate workspace produces a persistent result of only a few kilobytes.

Validation requirements

Focused tests must cover:

- exact synthetic weak decompositions;
- bounded negative results;
- source-incomplete and resource-truncated outcomes;
- matrix/provenance memory guards;
- deterministic candidate digests;
- independent rejection of forged weak witnesses even after JSON redigesting;
- serialization and catalog verification; and
- LTA orbit representatives.

References

1. W. Goetzke and H.-J. Klein, *Properties and Efficient Algorithmic Determination of Different Classes of Rings in Finite and Infinite Polyhedral Networks*, J. Non-Cryst. Solids **127**, 215–220 (1991), DOI: 10.1016/0022-3093(91)90145-V.
2. X. Yuan and A. N. Cormack, *Efficient Algorithm for Primitive Ring Statistics in Topological Networks*, Comput. Mater. Sci. **24**, 343–360 (2002), DOI: 10.1016/S0927-0256(01)00256-7.
3. V. A. Blatov, O. Delgado-Friedrichs, M. O’Keeffe, and D. M. Proserpio, *Three-Periodic Nets and Tilings: Natural Tilings for Nets*, Acta Cryst. A **63**, 418–425 (2007), DOI: 10.1107/S0108767307038287.